

Detecting Anomaly in ECG

Presented By:

1905040 - Asif Al Shahriar

1905046 - Niaz rahman

Group: A2_5





Problem Statements

- The project's goal is to create a system that
 - tracks heart activity
 - alerts of any irregularities
- The project addresses
 - A real-time, automated, and reliable solution using time series analysis
 - Identification of irregular patterns in sequential data of heart

Related Paper

- <https://www.sciencedirect.com/science/article/abs/pii/S0952197623006681>
- <https://www.mdpi.com/1999-4893/16/3/152>



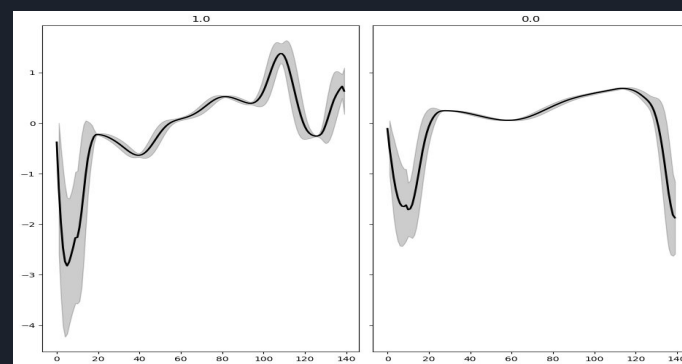
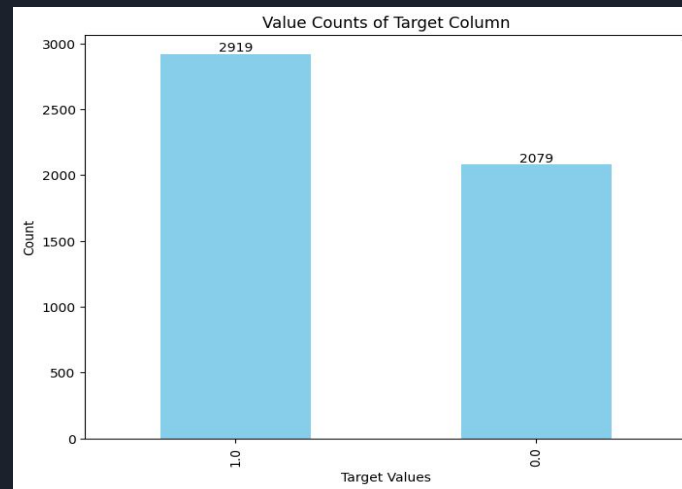
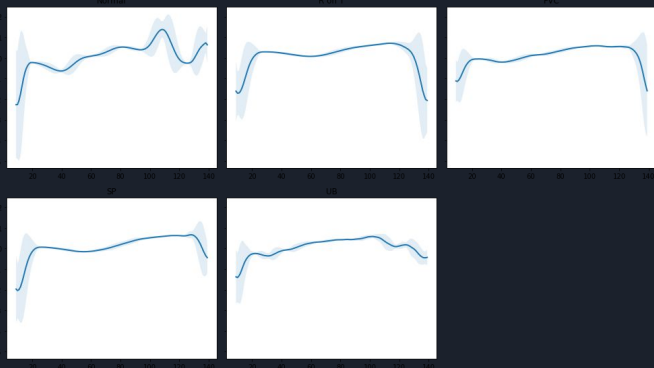
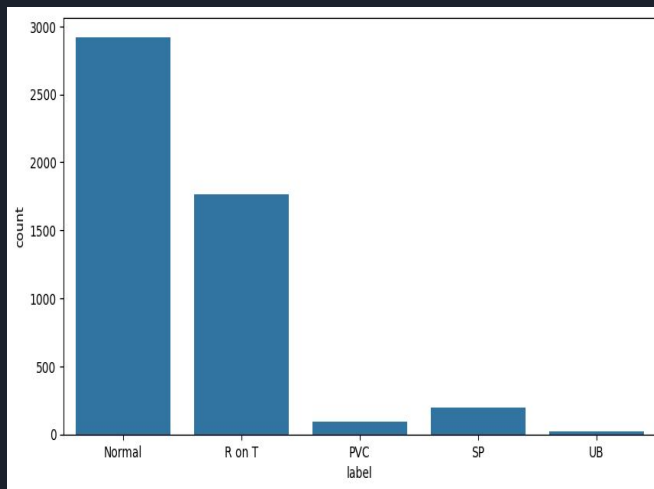
Dataset

<http://storage.googleapis.com/download.tensorflow.org/data/ecg.csv>

Dataset Overview

- The dataset is a 20-hour long ECG
- The data was pre-processed in two steps:
 - extract each heartbeat,
 - make each heartbeat equal length using interpolation.
- This data set contains 5000 ECGs, each with 140 data points.
- There are only two labels of 0 and 1
 - 0 means abnormal
 - 1 means normal

Dataset Overview





Development Method (Proposed Solution)

- **Data Preparation and Preprocessing:**
 - Normalizing the data
 - Segmenting it into readable sequences
 - Ensuring that it is free from noise
- **Model Architecture - Autoencoder LSTM:**
 - Encoding stage: Compress the input data into a condensed representation
 - Decoding stage: Reconstruct the data from this representation
 - LSTM : Process time-series data effectively.
 - Autoencoder : Employed for anomaly detection in sequential data.
- **Training with Normal ECG Data:**
 - Enables the model to learn a baseline of what constitutes a normal heart rhythm.
- **Real-time Anomaly Detection and Alert System:**
 - continuously analyzes incoming ECG data
 - focus is on the reconstruction error: significant deviations from the learned normal patterns are flagged as anomalies

Development Method (Proposed Solution)

BEGIN

/* Training Phase */

FOR each sample in normal_ECG_training_set DO

 encoded_ECG = LSTM_encoder(sample)

 reconstructed_ECG = LSTM_decoder(encoded_ECG)

 loss = compute_reconstruction_loss(sample, reconstructed_ECG)

 store_loss(loss)

 plot_loss(loss)

END FOR

FOR each sample in normal_ECG_validation_set DO

 encoded_ECG = LSTM_encoder(sample)

 reconstructed_ECG = LSTM_decoder(encoded_ECG)

 validation_loss = compute_reconstruction_loss(sample, reconstructed_ECG)

END FOR

/* Testing Phase */

FOR each sample in (normal_ECG_test_set U anomaly_ECG_test_set) DO

 reconstructed_ECG = LSTM_decoder(trained_model, sample)

 test_loss = compute_reconstruction_loss(sample, reconstructed_ECG)

 threshold = determine_threshold(method='Kapur')

 IF test_loss $\leq$ threshold THEN

 predicted_label = 'normal'

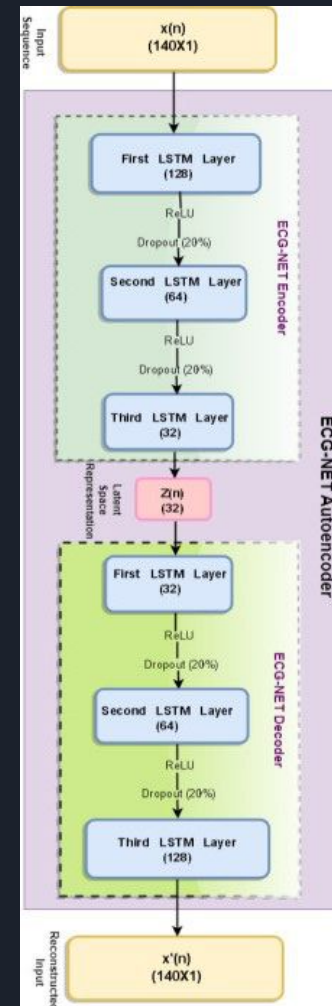
 ELSE

 predicted_label = 'anomaly'

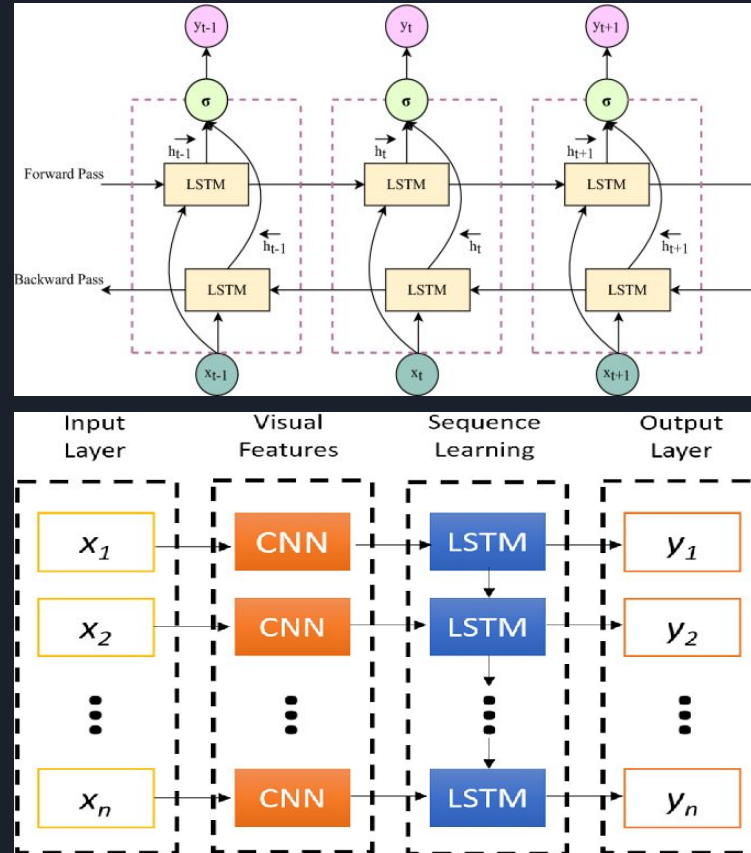
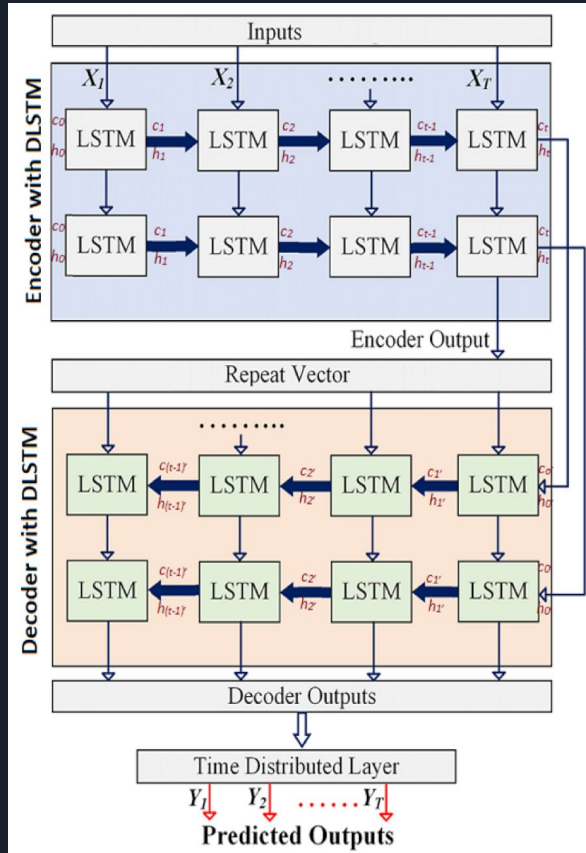
 END IF

END FOR

END



Development Method (Proposed Future Solution)





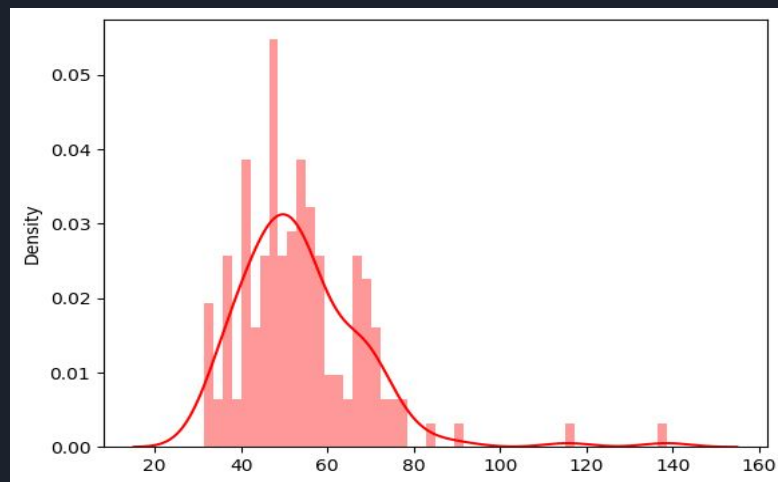
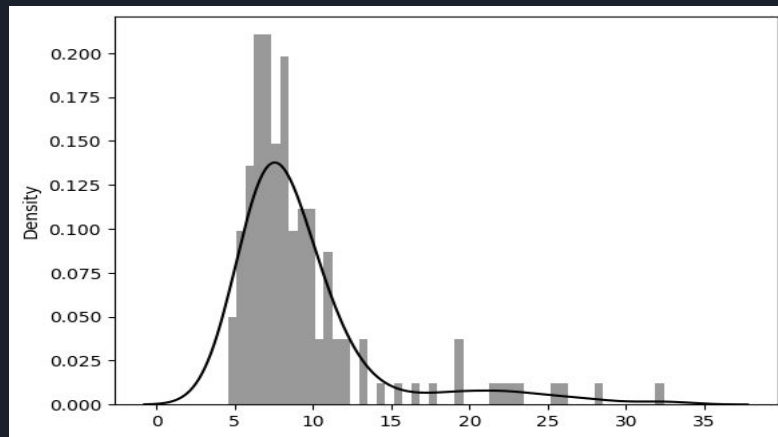
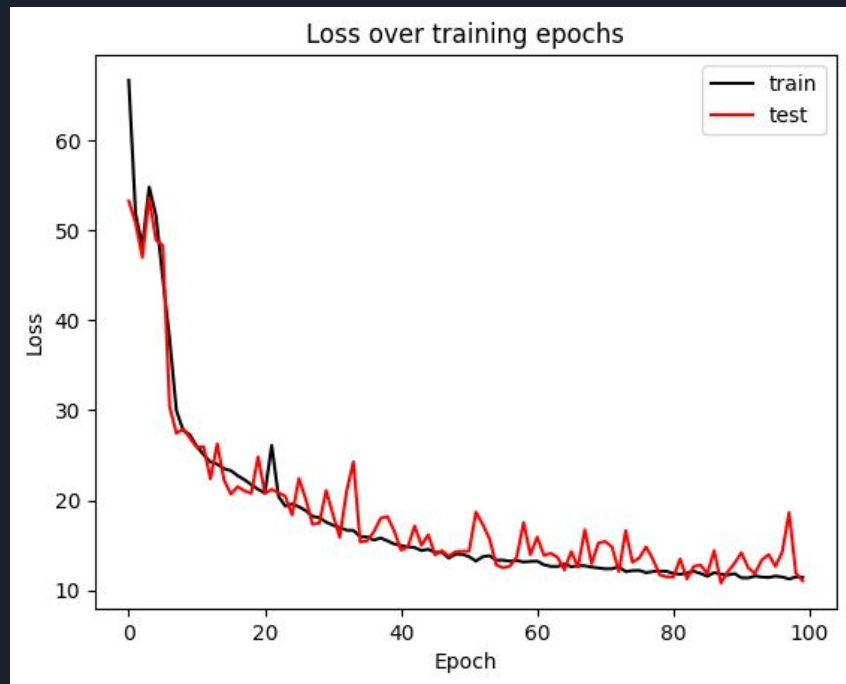
Loss Function and Its Intuition

L1 loss, also known as Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Robustness to Noise:** ECG signals are often contaminated with noise from various sources (e.g., motion artifacts, electrical interference). L1 loss, by measuring absolute differences, is less sensitive to outliers and noise compared to L2 loss.
- **Simplicity in Interpretation:** The absolute error calculated by L1 loss provides a straightforward interpretation of how far predictions deviate from actual classifications.
- **Temporal Context Preservation:** LSTM networks are designed to capture temporal dependencies in sequential data like ECG signals. When combined with L1 loss, the model can effectively learn to identify patterns over time while maintaining robustness against abrupt changes or noise within individual heartbeats.
- **Facilitating Model Training:** The optimization landscape created by L1 loss tends to be simpler than that of squared losses, potentially leading to faster convergence during training.

Loss Function Result





Performance Metrics

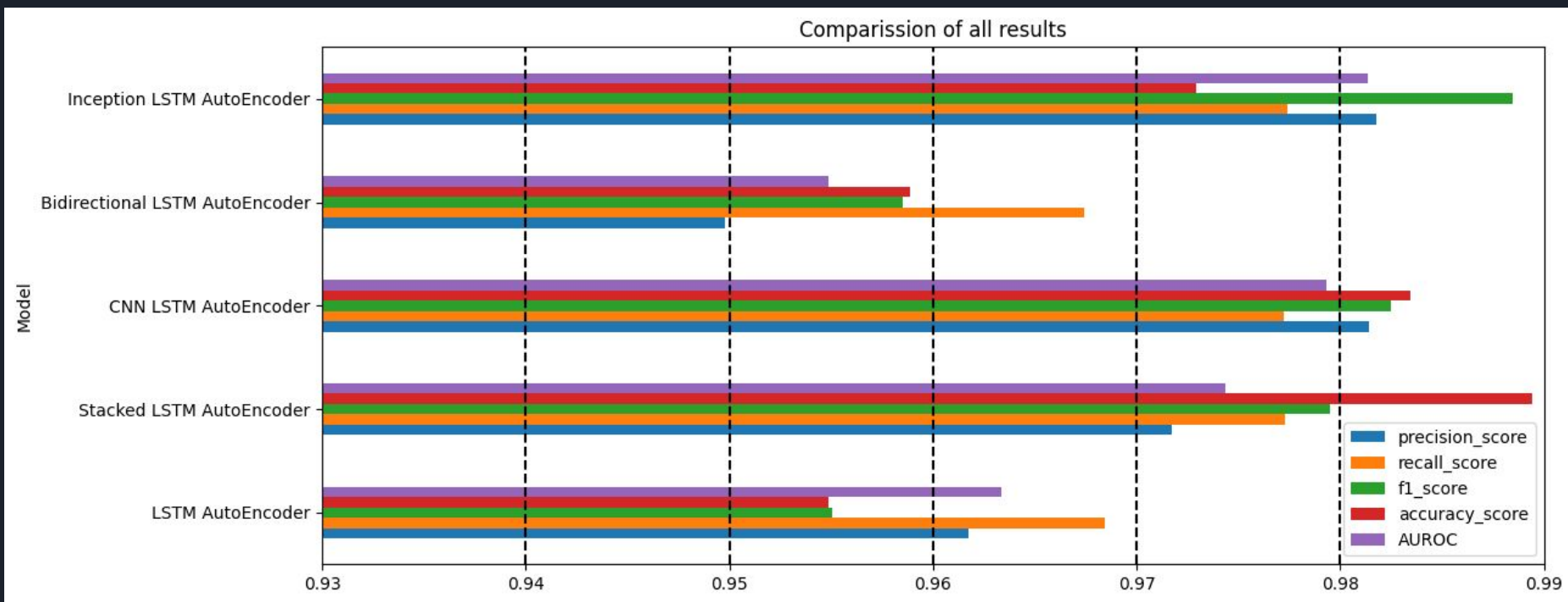
- **Reconstruction Loss :**
 - Visualized using a distribution plot on training data
 - A threshold is set to differentiate between normal and anomalous readings based on the loss.
 - Anomalies expected to have higher reconstruction losses
- Number of correctly identified **normal** sequences
- Number of correctly identified **anomalies**
- **Precision**
- **Recall**
- **F1-Score**
- **Plot Prediction :**
 - A function that visualizes the original and reconstructed ECG sequences, along with their reconstruction loss
 - Understanding the model's performance **on an individual sequence level.**



Performance Metrics Result

Model		precision_score	recall_score	f1_score	accuracy_score	AUROC
0	LSTM AutoEncoder	0.961734	0.968468	0.955057	0.954886	0.963334
1	Stacked LSTM AutoEncoder	0.971735	0.977273	0.979499	0.989453	0.974354
2	CNN LSTM AutoEncoder	0.981435	0.977254	0.982499	0.983453	0.979311
3	Bidirectional LSTM AutoEncoder	0.949772	0.967442	0.958535	0.958904	0.954891
4	Inception LSTM AutoEncoder	0.981772	0.977422	0.988454	0.972904	0.981347

Performance Metrics Result

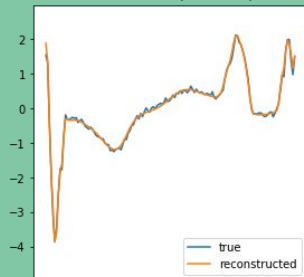


Performance Metrics Result

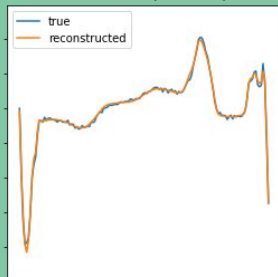
LSTM with Single Recurrent Layer

Threshold for anomaly detection: 0.1863

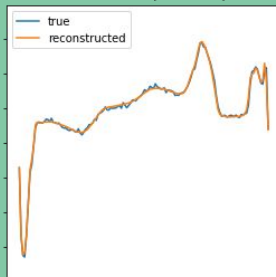
True Positive (loss: 0.05)



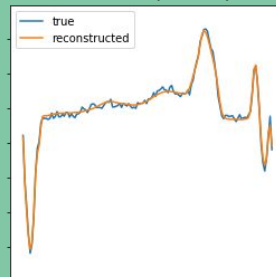
True Positive (loss: 0.05)



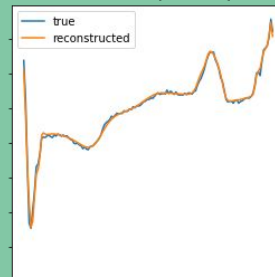
True Positive (loss: 0.05)



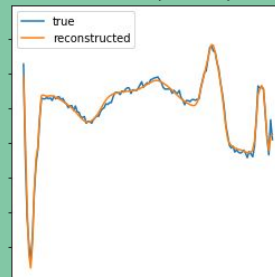
True Positive (loss: 0.07)



True Positive (loss: 0.05)



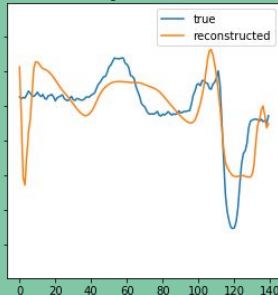
True Positive (loss: 0.09)



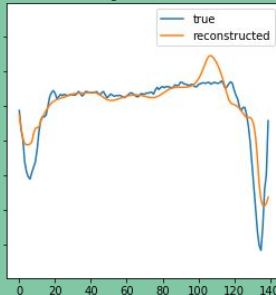
True Negative (loss: 0.3)



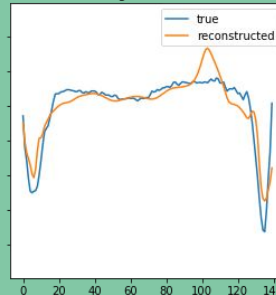
True Negative (loss: 0.59)



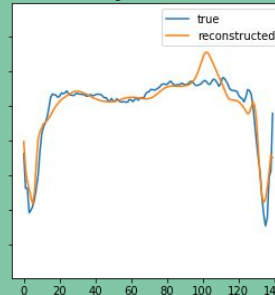
True Negative (loss: 0.29)



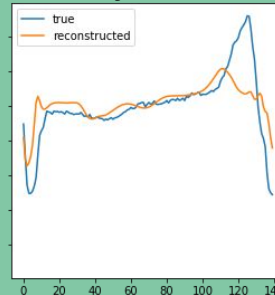
True Negative (loss: 0.3)



True Negative (loss: 0.24)



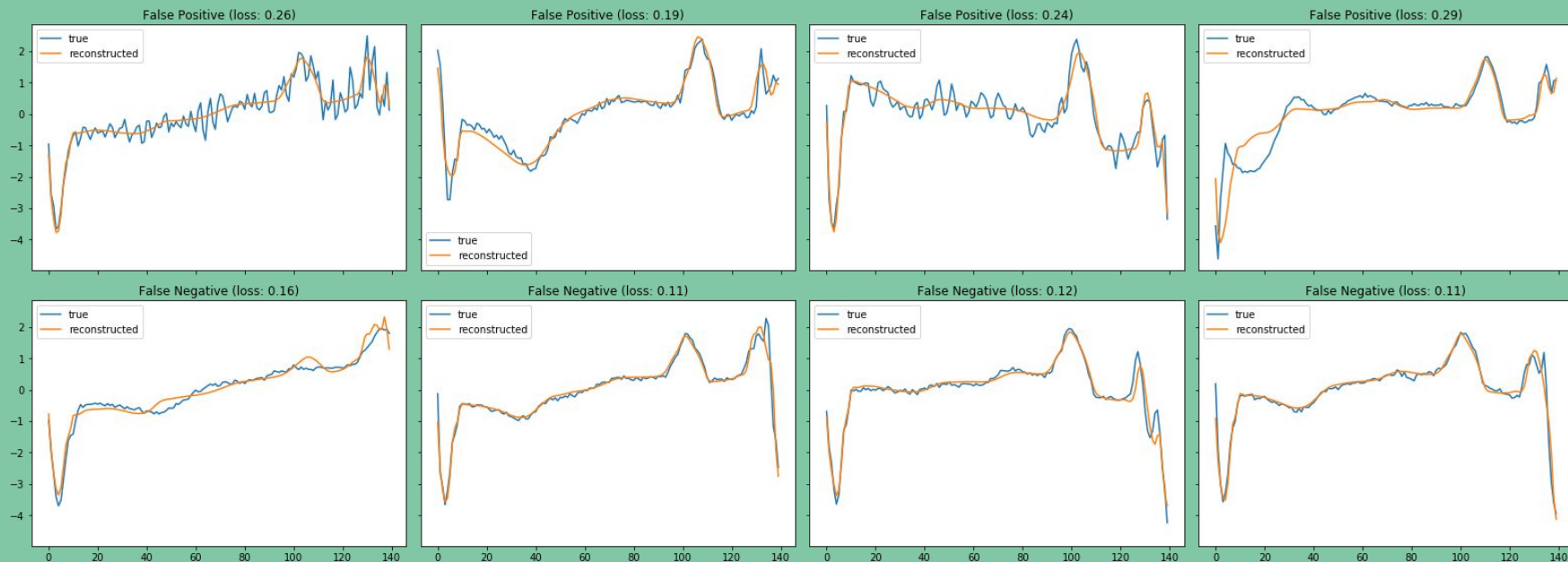
True Negative (loss: 0.45)



Performance Metrics Result

LSTM with Single Recurrent Layer

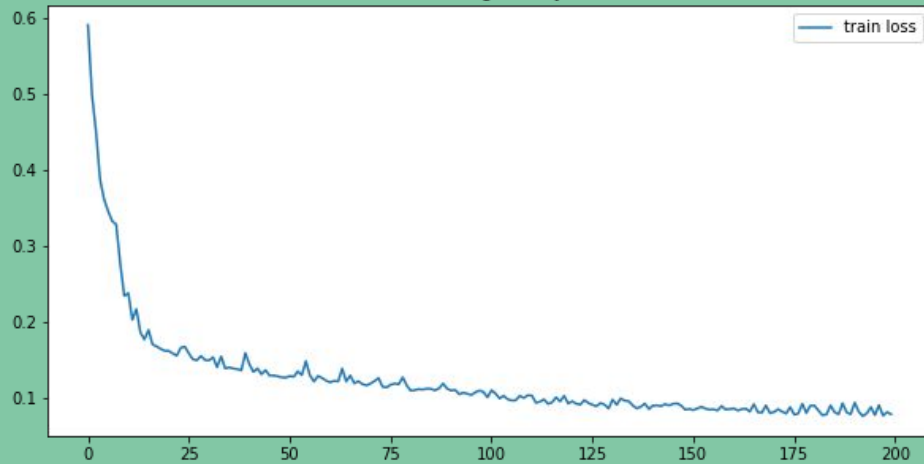
Threshold for anomaly detection: 0.1863



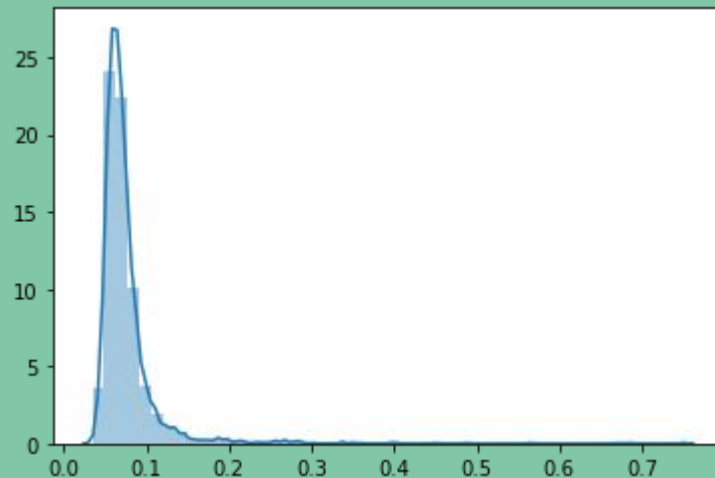
Performance Metrics Result

LSTM with Single Recurrent Layer

Training history



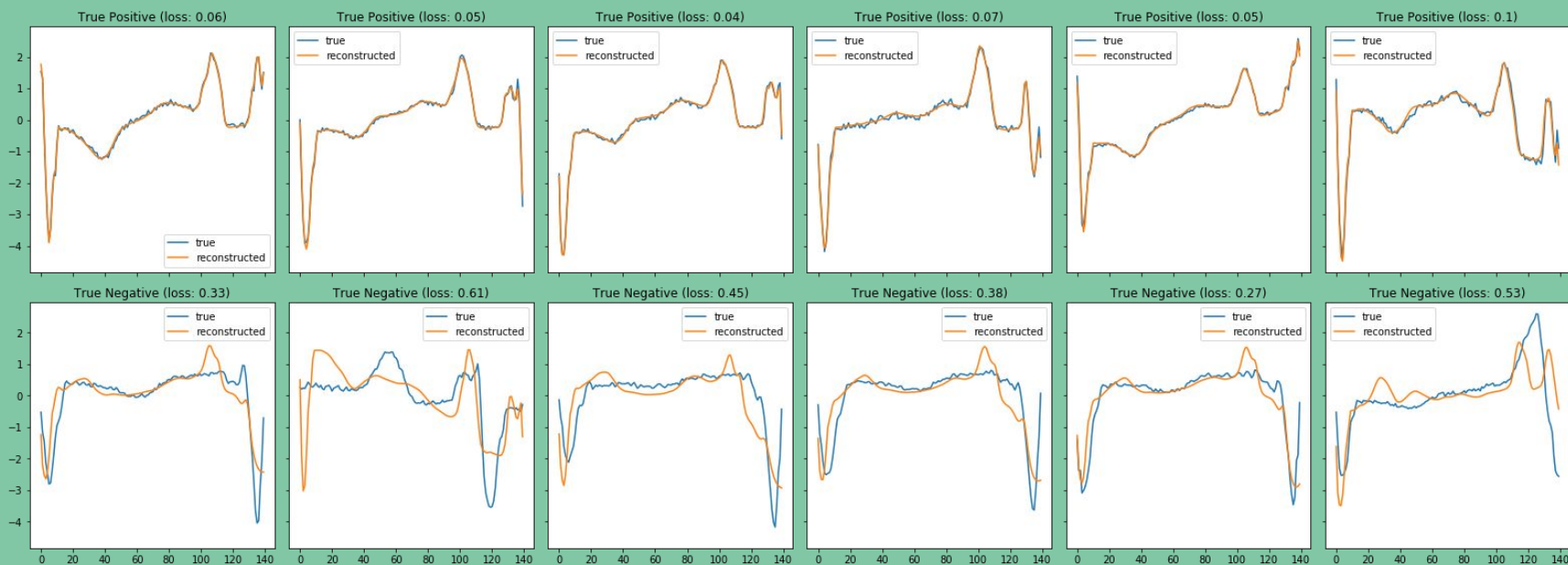
Train reconstruction losses distribution



Performance Metrics Result

Stacked LSTM with Double Recurrent Layer

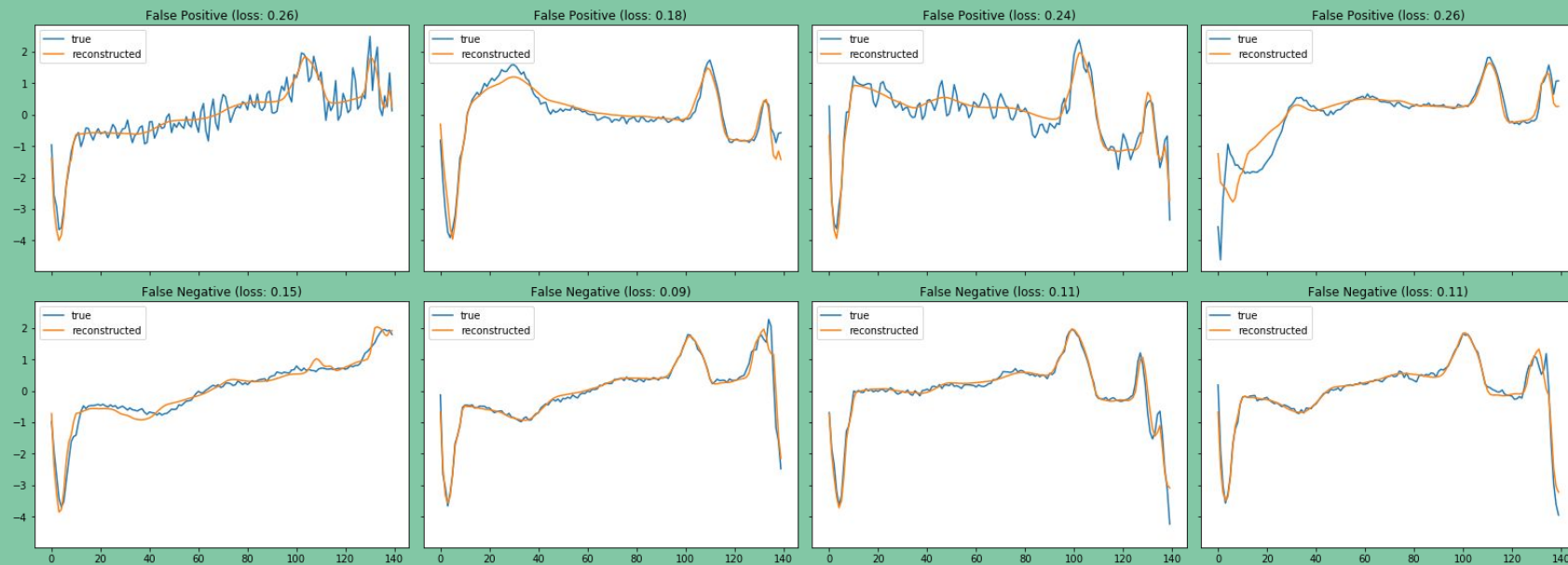
Threshold for anomaly detection: 0.17



Performance Metrics Result

Stacked LSTM with Double Recurrent Layer

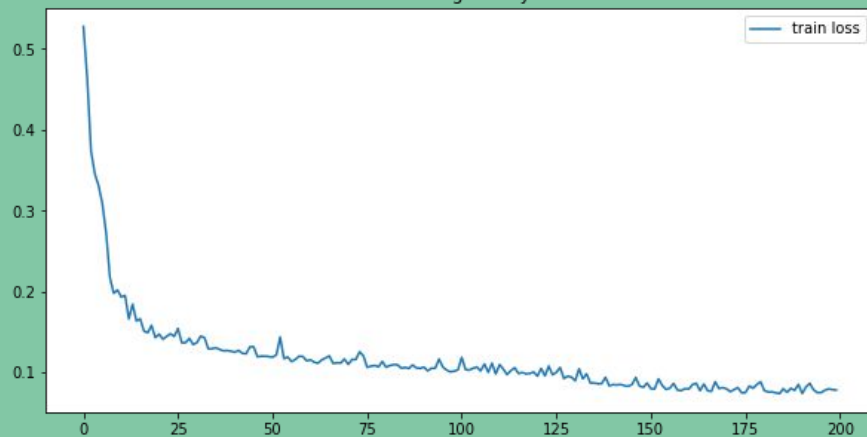
Threshold for anomaly detection: 0.17



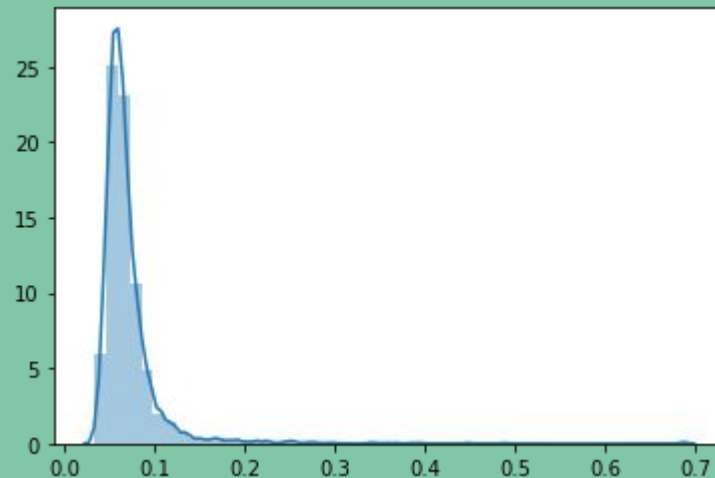
Performance Metrics Result

Stacked LSTM with Double Recurrent Layer

Training history



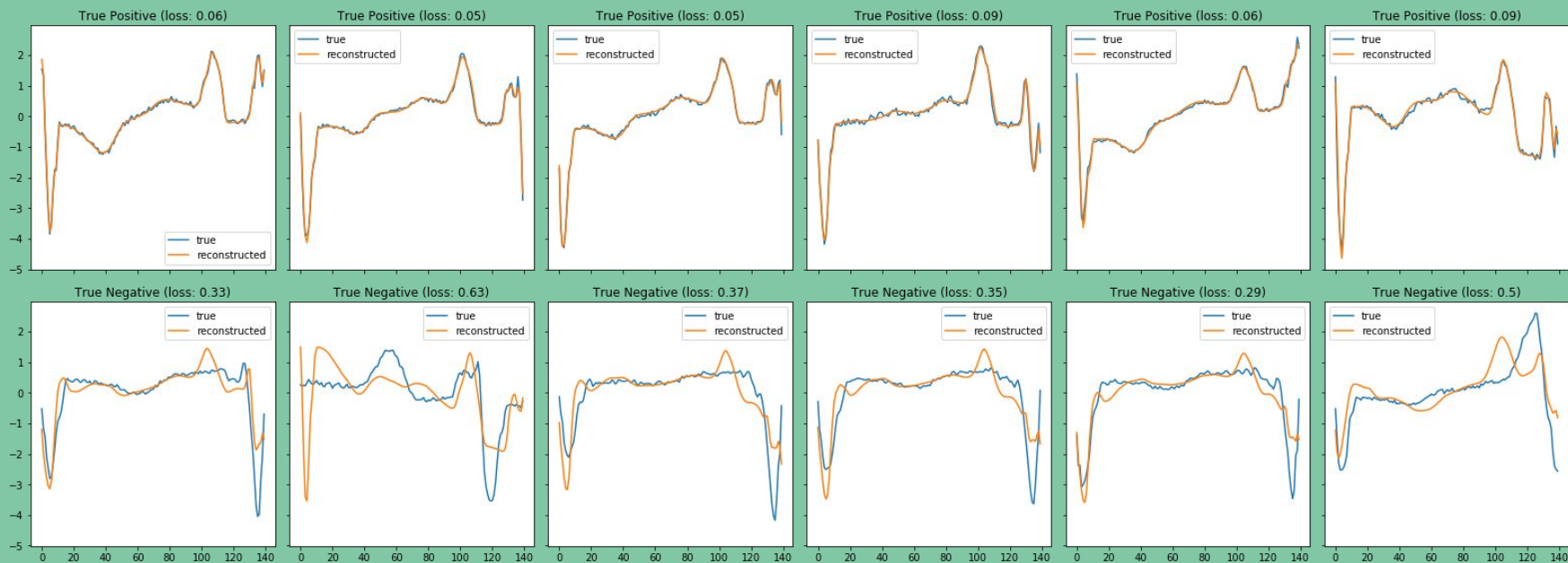
Train reconstruction losses distribution



Performance Metrics Result

CNN LSTM with Double Recurrent Layer

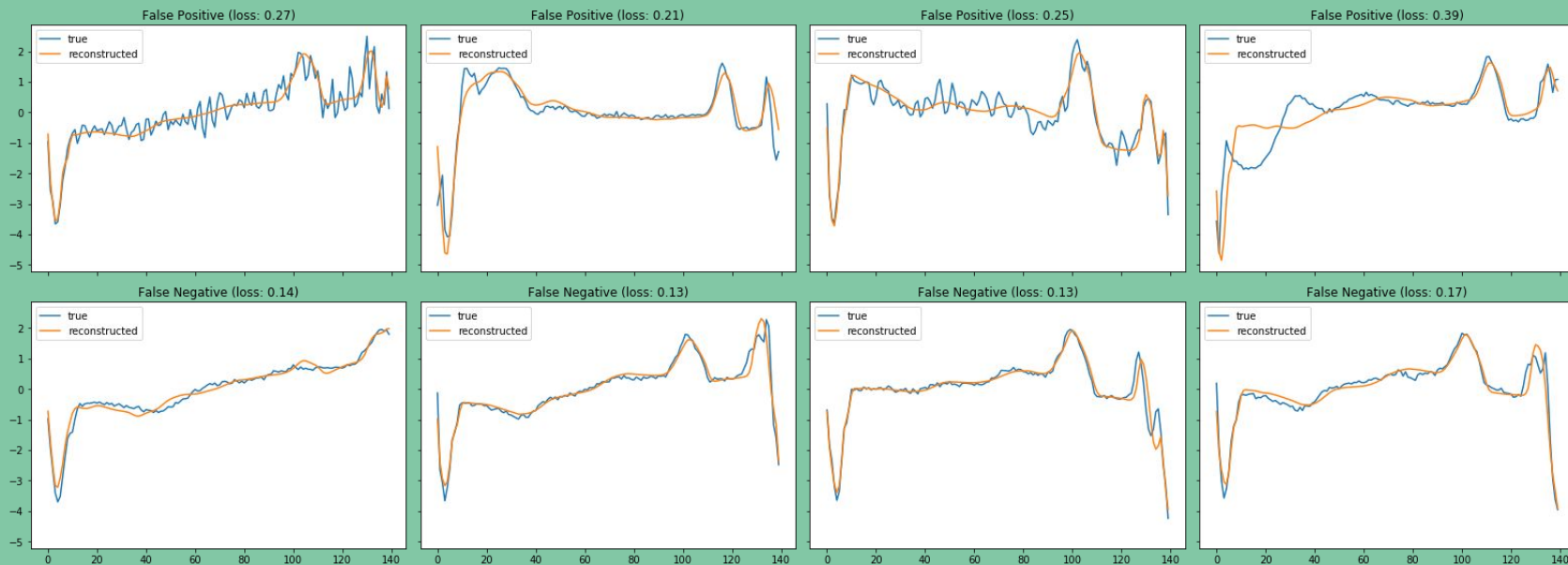
Threshold for anomaly detection: 0.1886



Performance Metrics Result

CNN LSTM with Double Recurrent Layer

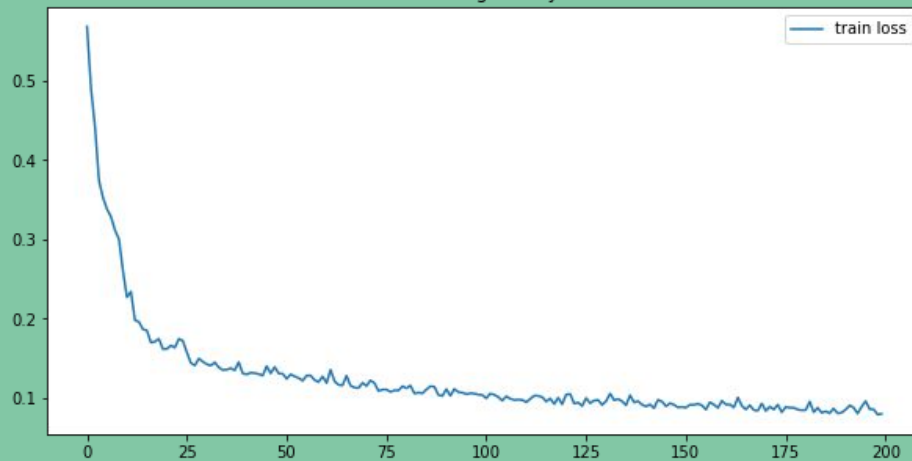
Threshold for anomaly detection: 0.1886



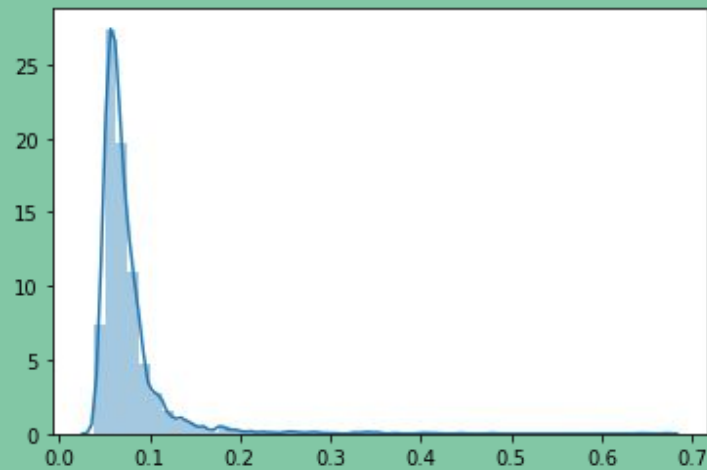
Performance Metrics Result

CNN LSTM with Double Recurrent Layer

Training history



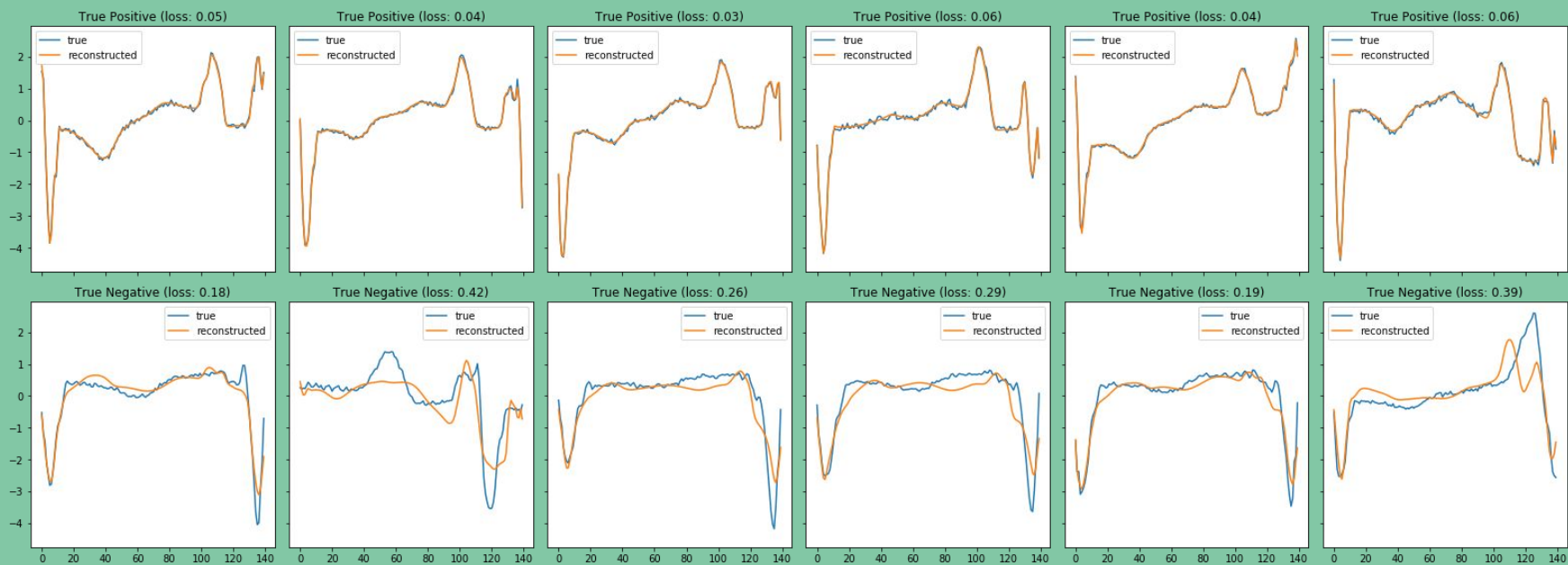
Train reconstruction losses distribution



Performance Metrics Result

Bidirectional LSTM with Double Recurrent Layer

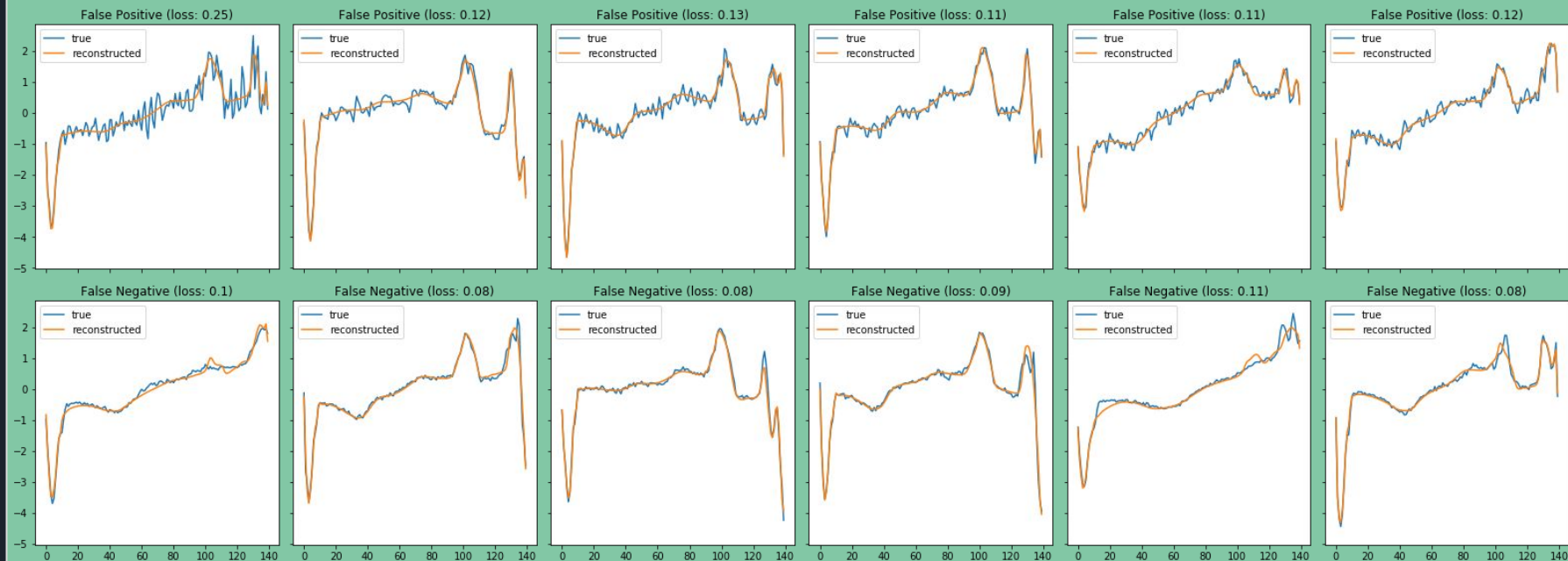
Threshold for anomaly detection: 0.1109



Performance Metrics Result

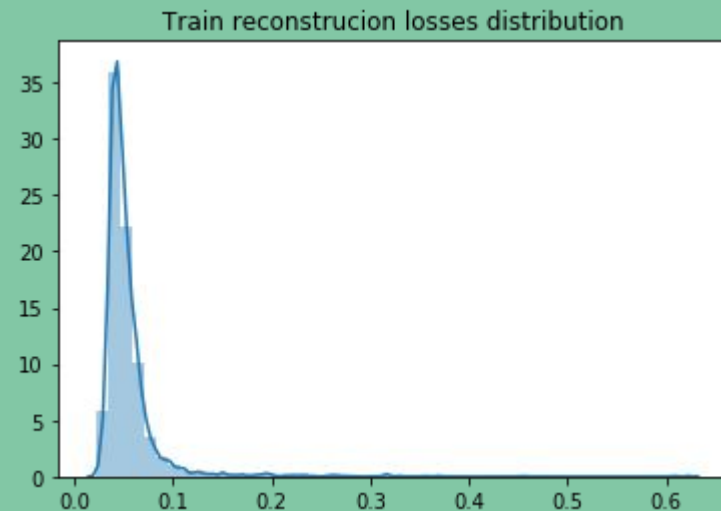
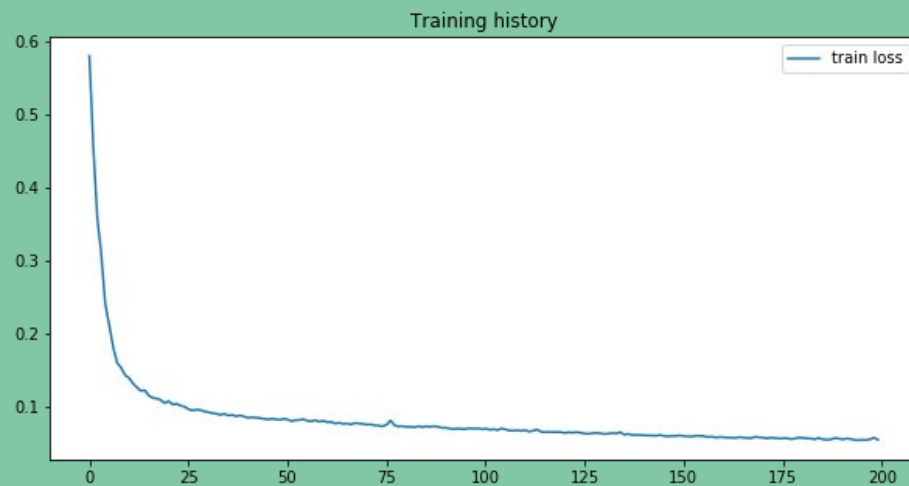
Bidirectional LSTM with Double Recurrent Layer

Threshold for anomaly detection: 0.1109



Performance Metrics Result

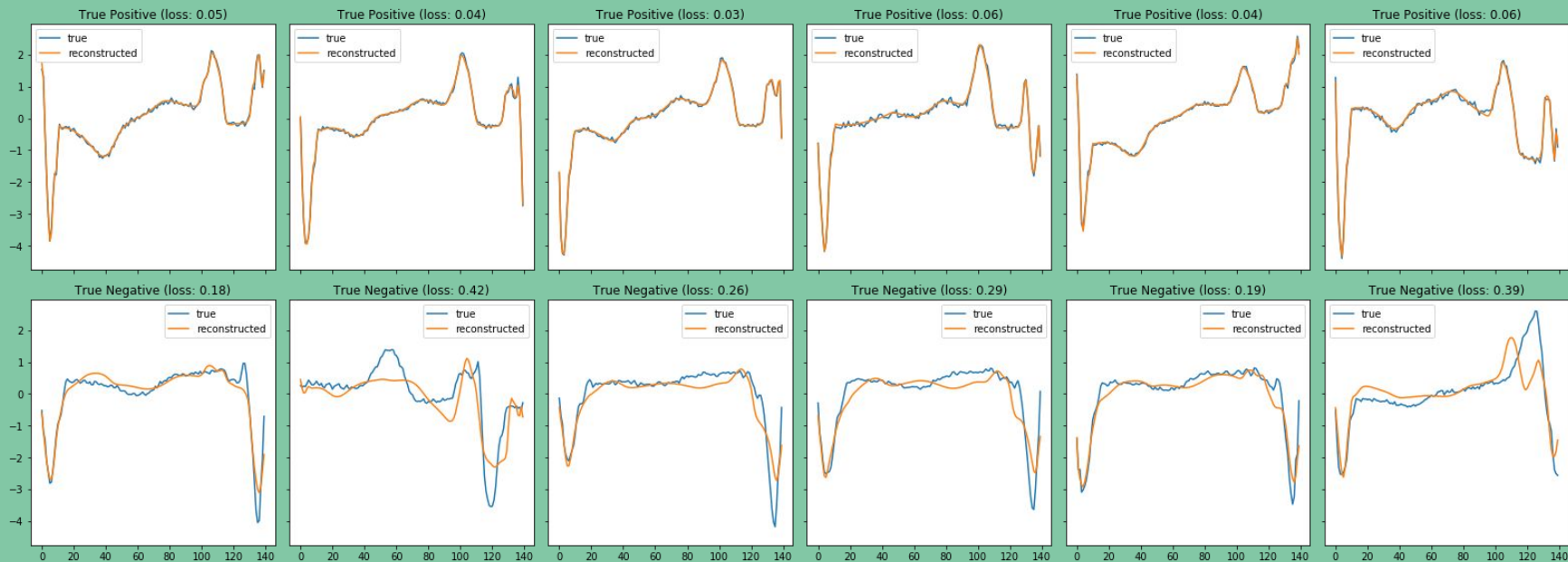
Bidirectional LSTM with Double Recurrent Layer



Performance Metrics Result

Inception LSTM with Double Recurrent Layer

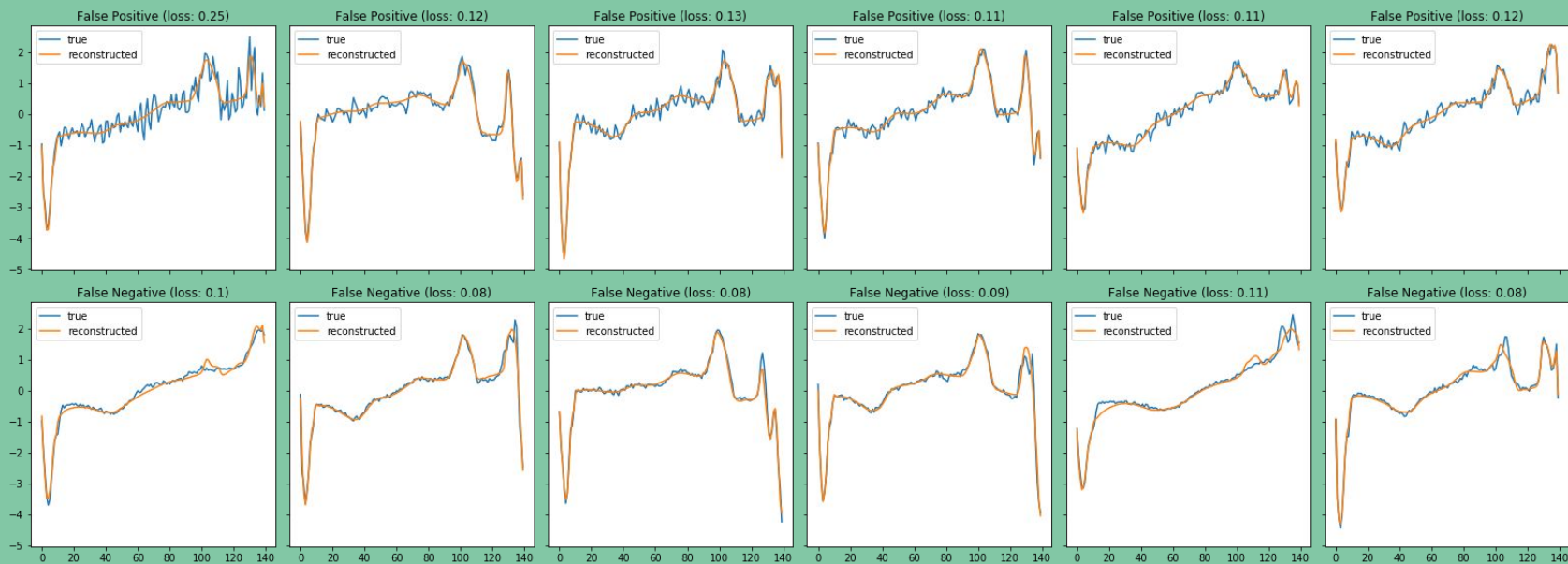
Threshold for anomaly detection: 0.1109



Performance Metrics Result

Inception LSTM with Double Recurrent Layer

Threshold for anomaly detection: 0.1109



Performance Metrics Result

Inception LSTM with Double Recurrent Layer

